

Micro-controller architecture -8051, PIC, and ARM

Harvard and Von Neumann architectures

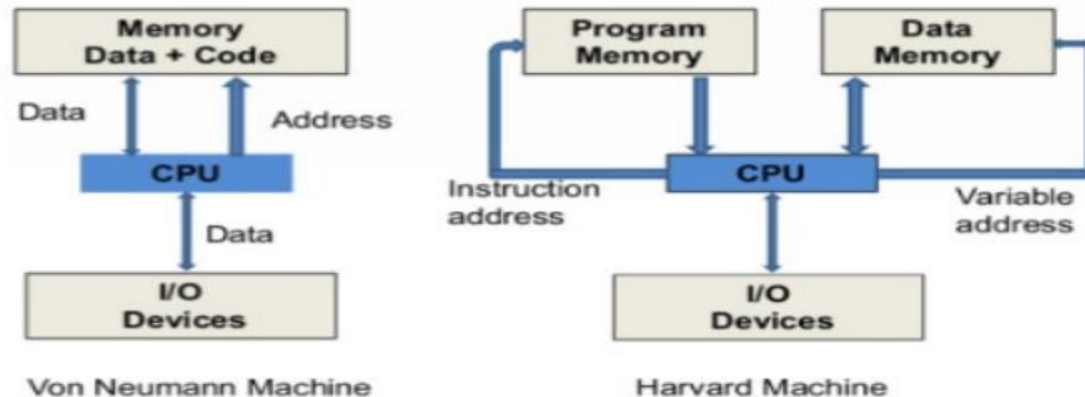
Von Neumann Architecture: Princeton architecture

Von Neumann Architecture is a digital computer architecture whose design is based on the concept of **stored program computers** where program data and instruction data are stored in the same memory. This architecture was designed by the famous mathematician and physicist **John Von Neumann** in 1945.

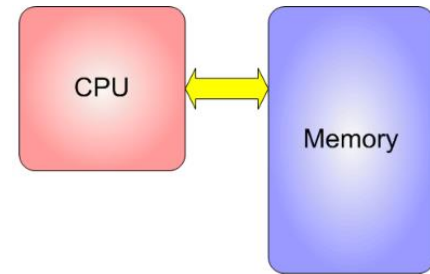
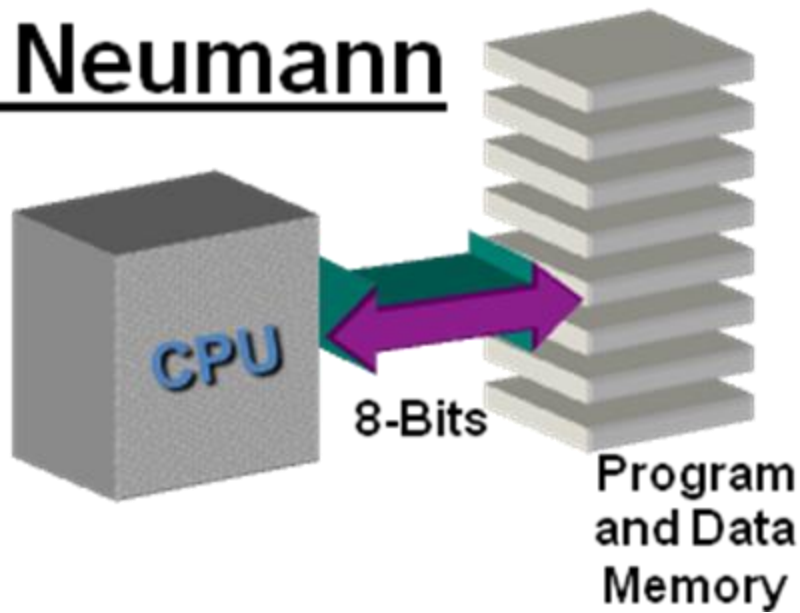
Data can pass through bus in half duplex mode to and from CPU.

Harvard Architecture:

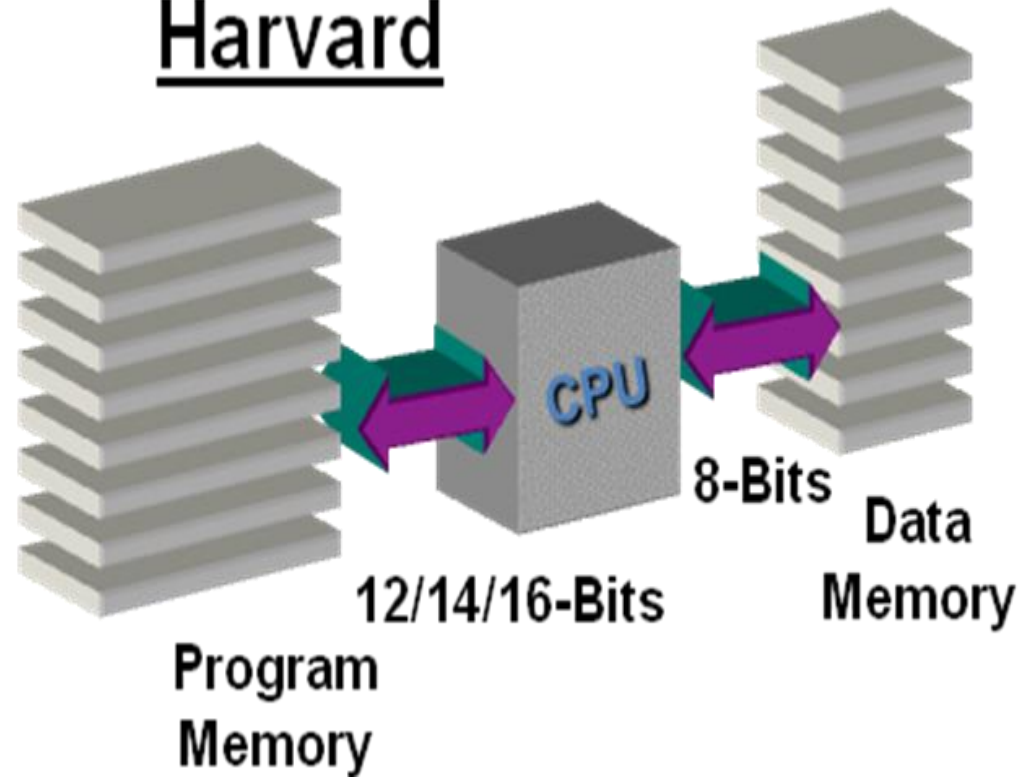
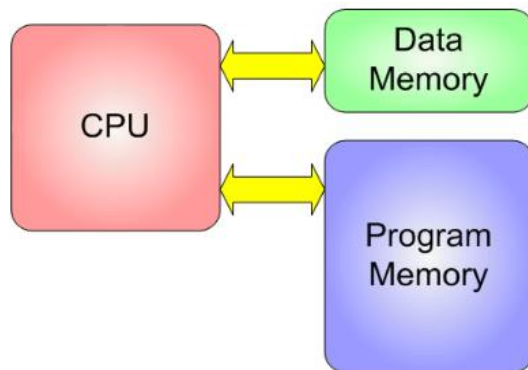
Harvard Architecture is the digital computer architecture whose design is based on the concept where there are separate storage and separate buses (signal path) for instruction and data. It was basically developed to overcome the bottleneck of Von Neumann Architecture.



Von Neumann

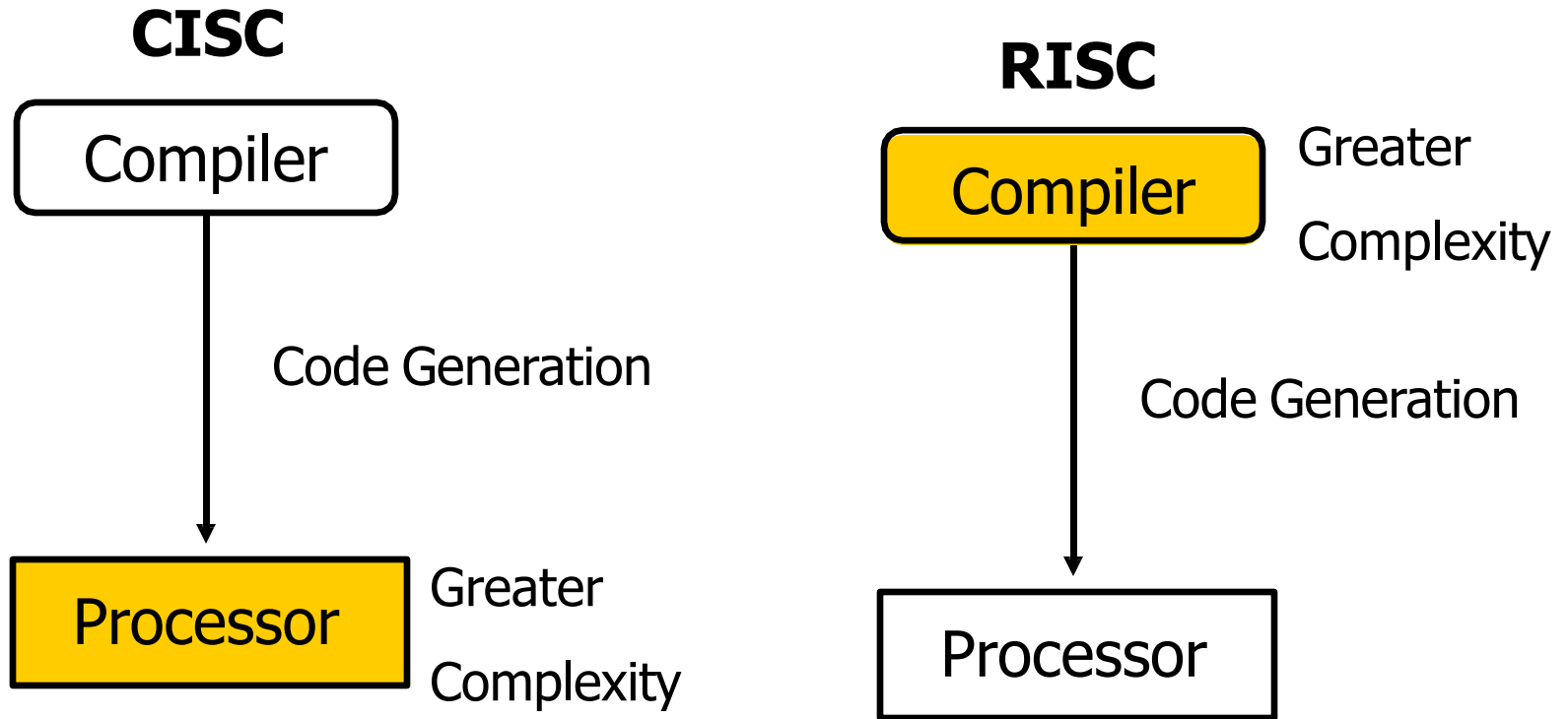


Harvard



RISC	CISC
Fixed width instructions	Variable length instructions
Few formats of instructions	Several formats of instructions
Load/Store Architecture	Memory values can be used as operands in instructions
Large Register bank	Small Register Bank
Instructions are pipelinable	Pipelining is Complex
Hardwired instruction decode	Microcode ROMS instruction decoder
Single cycle execution of all instructions	Multi cycle execution on instruction

CISC vs. RISC



Why Is RISC Important? ... Because of the **reduced instruction set and simple decoding logic**, less chip space is used, fewer transistors are required, and more general-purpose registers can fit into the central processing unit.

ARM processor

➤ An ARM processor is one of a family of CPUs based on the RISC (reduced instruction set computer) architecture developed by Advanced RISC Machines (ARM). ARM makes 32-bit and 64-bit RISC multi-core processors.

ARM processors are extensively used in consumer electronic devices such as [smartphones](#), [tablets](#), multimedia players and other mobile devices, such as [wearables](#). Because of their reduced [instruction set](#), they require fewer [transistors](#), which enables a smaller die size for the integrated circuitry (IC). The ARM processor's smaller size, reduced complexity and lower power consumption makes them suitable for increasingly miniaturized devices.

ARM processor features include:

Load/store architecture.

An [orthogonal](#) instruction set.

Mostly single-cycle execution.

Enhanced power-saving design.

64 and 32-bit execution states for scalable high performance.

[Hardware virtualization](#) support.

The simplified design of ARM processors enables more efficient multi-core processing and easier coding for developers. While they don't have the same raw compute [throughput](#) as the products of [x86](#) market leader Intel, ARM processors sometimes exceed the performance of Intel processors for applications that exist on both architectures.

Features of ARM Processor {ARM7TDMI}

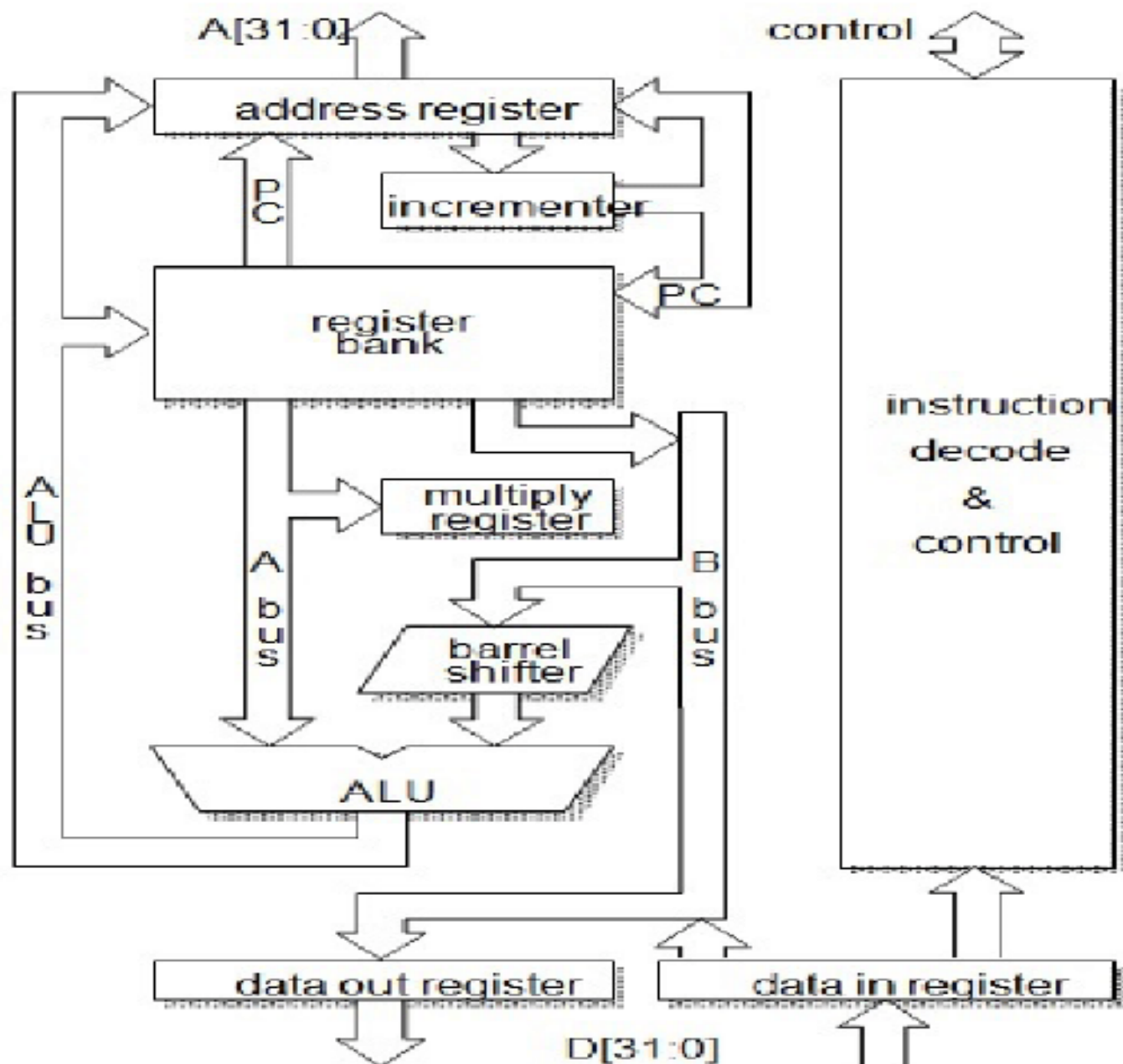
❖ Introduction

- ❑ Originally developed by Arcon Computers in mid 1980s then renamed to ARM {Advanced RISC Machine}.
- ❑ 'ARM Family' has varieties of 32bits controllers used in many commercial applications, Embedded Systems, Smart Phones (Cortex A), Printers (Cortex R), Medical Instruments (Cortex M), etc.
- ❑ ARM processor accounts more then 75% of 32bits of processors in market share of industry in 2009.

❖ Features of ARM7

- ❑ ARM7 has 32 bits of MCU & ALU.
- ❑ ARM7 has 32 bits of Data bus with aligned memory space. {Means with each machine cycle it will execute 32bits via data bus. Here aligned addressing is done.}
- ❑ ARM7 has all the instructions with 32 bits.
- ❑ ARM7 has 32 bits of Address Bus. {So we can interface 4GB memory with it directly.}
- ❑ ARM7 follows Von Neuman Model. {So 4GB memory will be common for code and data.} with latest versions they have switched to Harvard Architecture.

- ❑ ARM7 has three stage Pipelining
 - Fetch
 - Decode
 - Execute
- ❑ ARM7 has 37 registers of 32 bits. At time 16 registers available {R0 – R15}.
- ❑ ARM7 has LOAD – STORE Architecture. {Means for LOAD and STORE operations we have to use separate instructions.}
- ❑ ARM7 has 7 operating Modes.
- ❑ ARM7 has 7 interrupts/Exceptions.
- ❑ ARM7 has 7 Addressing Modes.
- ❑ ARM7 data formats {8bits – Byte, 16bits – Half word, 32bits - Word}
- ❑ ARM7TDMI
 - T – Thumb Instructions
 - D – Hardware Debugging
 - M – Long Multiply instruction
 - I – ICE Microcells for Breakpoints and watch points in program.



The ARM architecture processor is an advanced reduced instruction set computing [RISC] machine and it's a 32bit reduced instruction set computer (RISC) microcontroller. It was introduced by the Acron computer organization in 1987. This ARM is a family of microcontroller developed by makers like ST Microelectronics, Motorola, and so on. The ARM architecture comes with totally different versions like ARMv1, ARMv2, etc., and, each one has its own advantage and disadvantages.

It has the various blocks

❑ **Arithmetic Logic Unit**

❑ **Booth multiplier**

❑ **Barrel shifter**

❑ **Control unit**

❑ **Register file**

It has flags and a register bank .

Arithmetic Logic Unit (ALU)

The ALU has two 32-bits inputs. The primary comes from the register file, whereas the other comes from the shifter. Status registers flags are modified by the ALU outputs.

Booth Multiplier Factor

The multiplier has 32-bit inputs and the inputs return from the register file. The entity representation of the multiplier factor is shown in the above block diagram. Booth algorithm is a noteworthy multiplication algorithmic rule for 2's complement numbers. This treats positive and negative numbers uniformly. Moreover, the runs of 0's or 1's within the multiplier factor are skipped over without any addition or subtraction being performed, thereby creating possible quicker multiplication.

Barrel Shifter

The barrel shifter features a 32-bit input to be shifted. This input is coming back from the register file or it might be immediate data. The shifter has different control inputs coming back from the instruction register. The Shift field within the instruction controls the operation of the barrel shifter. This field indicates the kind of shift to be performed (logical left or right, arithmetic right or rotate right). The quantity by which the register ought to be shifted is contained in an immediate field within the instruction or it might be the lower 6 bits of a register within the register file.

The shift_val input bus is 6-bits, permitting up to 32 bit shift. The shift type indicates the needed shift sort of 00, 01, 10, 11 are corresponding to shift left, shift right, an arithmetic shift right and rotate right, respectively. The barrel shifter is especially created with multiplexers.

Control Unit

For any microprocessor, control unit is the heart of the whole process and it is responsible for the system operation, so the control unit design is the most important part within the whole design. The control unit is sometimes a pure combinational circuit design. Here, the control unit is implemented by easy state machine. The processor timing is additionally included within the control unit. Signals from the control unit are connected to each component within the processor to supervise its operation.

Processor Modes

- The ARM has seven basic operating modes:
 - Each mode has access to own stack and a different subset of registers
 - Some operations can only be carried out in a privileged mode

Exception modes	Mode	Description		User	usr
	Supervisor (SVC)	Entered on reset and when a Software Interrupt instruction (SWI) is executed	Privileged modes	Fast interrupt	fiq
	FIQ	Entered when a high priority (fast) interrupt is raised		Interrupt	irq
	IRQ	Entered when a low priority (normal) interrupt is raised		Supervisor	svc
	Abort	Used to handle memory access violations		Abort	abt
	Undef	Used to handle undefined instructions		System	sys
	System	Privileged mode using the same registers as User mode		Undefined	und
	User	Mode under which most Applications / OS tasks run	Unprivileged mode		

- Processor modes determine which registers are active and access rights to cpsr register itself
- Each processor mode is either privileged or non privileged
- Privileged mode allows full read write access to cpsr
- Non-privileged mode allows read access to control field of cpsr but allows read write access to condition flags
- There are Total 7 processor modes
 - Six privileged modes (abort, fast interrupt request , interrupt request, supervisor, system and undefined)
 - One non-privileged mode (user)

Load Store Architecture

- Only load and store instructions access the memory, all other instructions use registers as operands
 - LDR R0, [R1] ADD R0, R1, R2
- The instructions will only process values which are in registers and will always place results of such processing into a register.
- The only operations which apply to memory are that copy memory values into registers (load instructions) or copy register values into memory (store instructions)

To help manage such situations, most processors have different operating modes that allow different levels of “privileged” access to areas of memory, system settings, peripheral devices, and other resources.

➤ Typically, the lowest priority mode is the “user” mode that is used by regular application/user programs. Processes running in user mode can be prevented from accessing certain areas of memory, certain registers, and certain peripherals. This limited access is essential to creating a stable, shared processing environment.

➤The most privileged mode is typically a “system” mode that allows a program to freely access all resources without restriction. This mode is typically used sparingly, and only when needed. Usually, only “System” software is allowed to run in system mode, and even then it only does so when strictly needed. System software typically manages user applications, and allocates shared resources like memory or data ports. System software might run mostly in user mode, and switch to system mode only when necessary to access a protected register, or reassign certain resources.

Refer- <https://www.realdigital.org/doc/6f6b0326edcd76735fc5f7a94637ea41>

The ARM has several operating modes that are used for various purposes:

➤ **User mode is the basic mode in which application programs run.** User mode is the only unprivileged mode, and it has restricted access to system resources. Typically, a processor spends more than 99% of its time in user mode.

➤ **System mode provides unrestricted access to all system resources.**

Supervisor mode can only be entered in certain controlled ways, and it is typically only entered when required to manage a particular resource.

➤ **Supervisor mode also provides unrestricted access to all system resources.** Supervisor mode is entered on reset or power-up, or when software executes a Supervisor Call instruction (SVC). Supervisor mode is similar to system mode, but offers access to a few more registers.

➤ **Abort mode is entered if a program attempts to access a non-existing memory location.**

Abort mode also offers access to a few private registers that other modes can't access.

Undefined mode is entered for any instruction-related exceptions, including any attempt to execute an unimplemented instruction.

➤ **IRQ mode is entered in response to a normal interrupt request from an external device.**

➤ **FIQ mode is entered in response to a fast interrupt request from an external device.** It is used to provide faster service for more urgent requests.

Monitor mode is available in some implementations to change between secure and non-secure states, and for debugging.

Hypervisor mode is available in some implementations to control certain virtualization extensions.

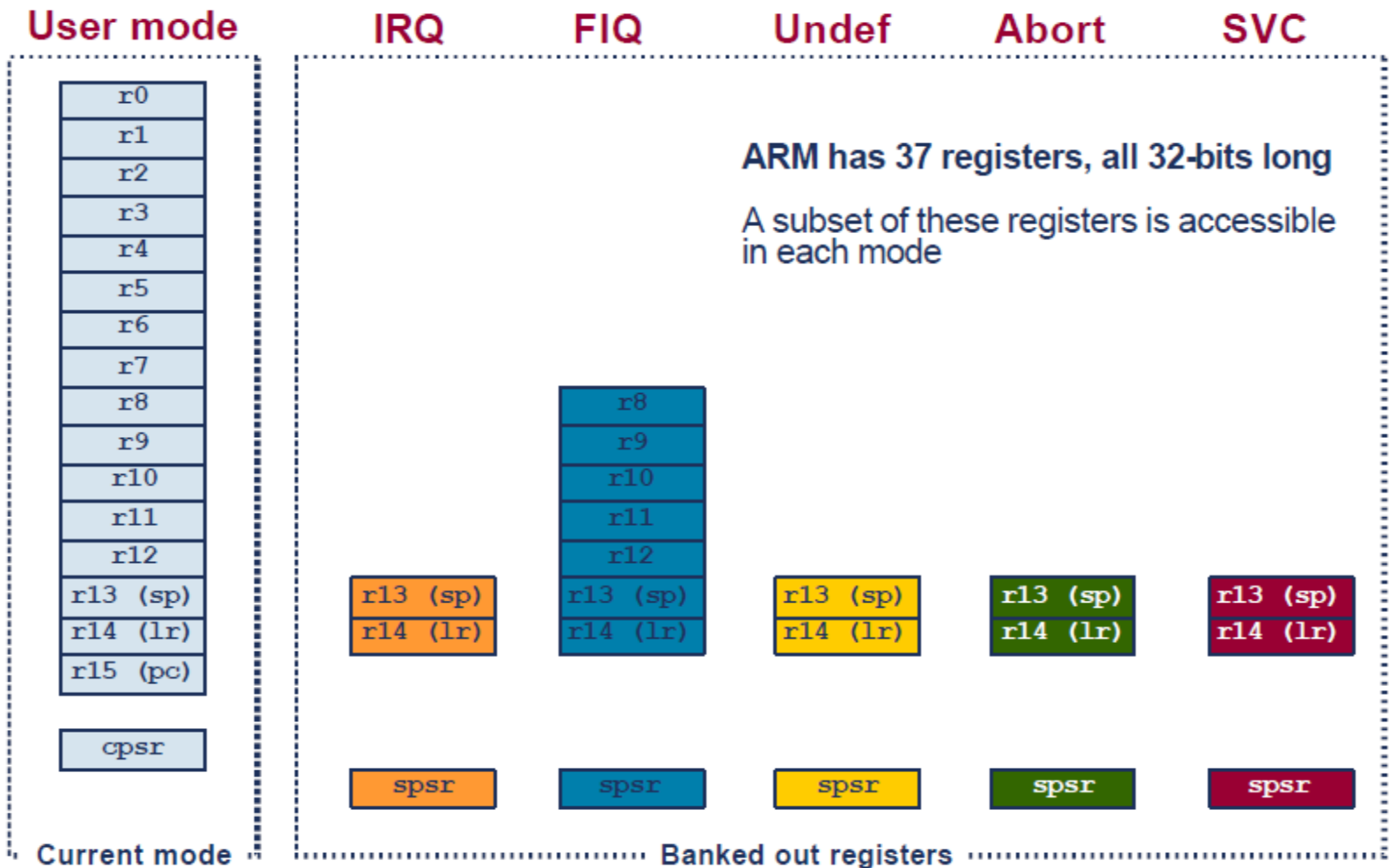
You can only enter **System mode from another privileged mode by modifying the mode bit of the Current Program Status Register (CPSR).**

- **ARM has 37 registers all of which are 32-bits long.**
 - 1 dedicated program counter
 - 1 dedicated current program status register (CPSR)
 - 5 dedicated saved program status registers (SPSR)
 - 30 general purpose registers
- **The current processor mode governs which of several banks is accessible. Each mode can access**
 - a particular set of **r0-r12** registers
 - a particular **r13** (the stack pointer, **sp**) and **r14** (the link register, **lr** –to store the return address in subroutine calls)
 - the program counter, **r15** (**pc**)
 - the current program status register, **cpsr**

Privileged modes (except System) can also access

- a particular **spsr** (saved program status register)

The ARM Register Set



The ARM7TDMI processor has a total of 37 registers:

31 general-purpose 32-bit registers (20-Invisible registers -16 general purpose –CPSR)
6 status registers.

These registers are not all accessible at the same time. The processor state and operating mode determine which registers are available to the programmer.

- All 37 registers in the register file.
- Of those, 20 registers are hidden from a program at different times. These registers are called *banked registers* and are identified by the shading in the diagram.
- They are available only when the processor is in a particular mode; for example, *abort* mode has banked registers *r13_abt*, *r14_abt* and *spsr_abt*. Banked registers of a particular mode are denoted by an underline character post-fixed to the mode mnemonic or *_mode*.

Refer-
<https://www.sciencedirect.com/topics/computer-science/banked-register>

Application View	User and system					
R0	<i>r0</i>					
R1	<i>r1</i>					
R2	<i>r2</i>					
R3	<i>r3</i>					
R4	<i>r4</i>					
R5	<i>r5</i>					
R6	<i>r6</i>					
R7	<i>r7</i>					
R8	<i>r8</i>					
R9	<i>r9</i>					
R10	<i>r10</i>					
R11	<i>r11</i>					
R12	<i>r12</i>					
R13	<i>r13 sp</i>					
SP	<i>r14 lr</i>					
LR	<i>r15 pc</i>					
PC						
APSR	<i>cpsr</i>					
	-					

		<i>Fast interrupt request</i>				
		<i>r8_fiq</i>				
		<i>r9_fiq</i>				
		<i>r10_fiq</i>				
		<i>r11_fiq</i>				
		<i>r12_fiq</i>				
		<i>r13_fiq</i>				
		<i>r14_fiq</i>				
		<i>Interrupt request</i>	<i>Supervisor</i>	<i>Undefined</i>	<i>Abort</i>	
		<i>r13_irq</i>	<i>r13_svc</i>	<i>r13_undef</i>	<i>r13_abt</i>	
		<i>r14_irq</i>	<i>r14_svc</i>	<i>r14_undef</i>	<i>r14_abt</i>	
		<i>spsr_fiq</i>	<i>spsr_irq</i>	<i>spsr_svc</i>	<i>spsr_undef</i>	<i>spsr_abt</i>

Refer-

<https://www.sciencedirect.com/topics/computer-science/banked-register>

Current Visible Registers

Abort Mode
SV

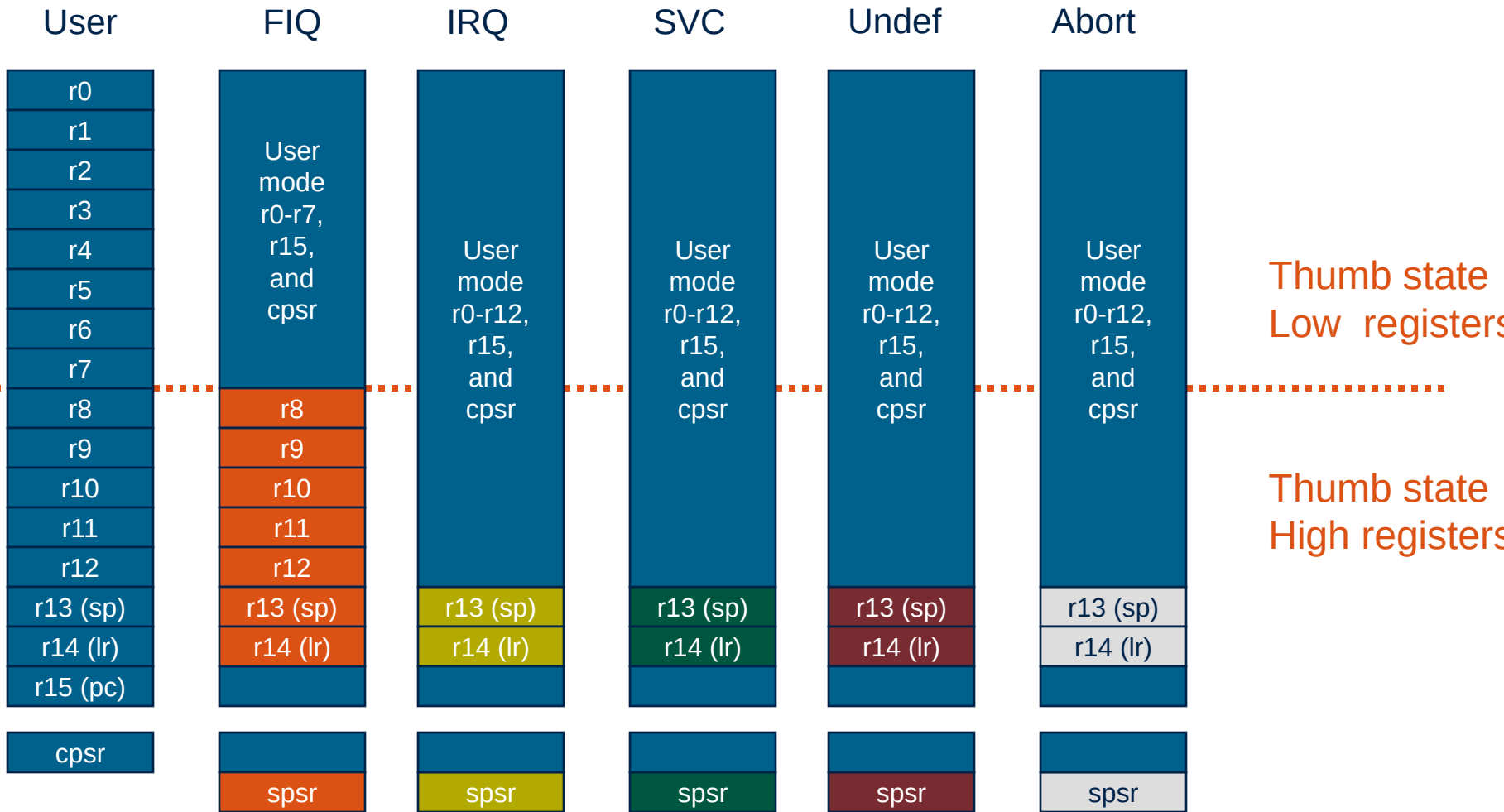
r0
r1
r2
r3
r4
r5
r6
r7
r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)
r15 (pc)

cpsr
spsr
spsr

All processor modes except *system* mode have a set of associated banked registers that are a subset of the main 16 registers. A banked register maps one-to-one onto a *user* mode register. If you change processor mode, a banked register from the new mode will replace an existing register.

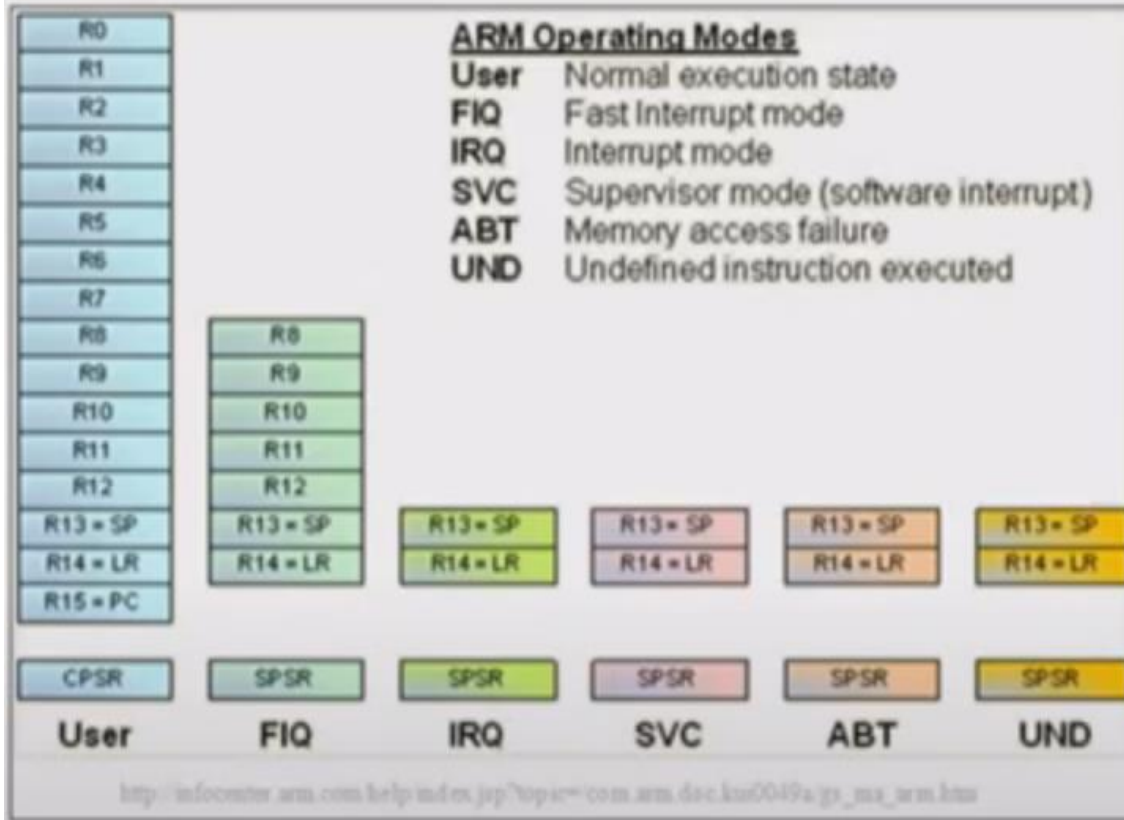
Banked out Registers

User	FIQ	IRQ	SVC	Undef
	r8			
	r9			
	r10			
	r11			
	r12			
r13 (sp)	r13 (sp)	r13 (sp)	r13 (sp)	r13 (sp)
r14 (lr)	r14 (lr)	r14 (lr)	r14 (lr)	r14 (lr)
	spsr	spsr	spsr	spsr
	spsr	spsr		spsr



Note: System mode uses the User mode register set

Processor modes



Processor States

1. ARM state
2. Thumb State
3. Jazzele State

- The ARM is a 32-bit architecture.
- When used in relation to the ARM:
 - **Byte** means 8 bits
 - **Halfword** means 16 bits (two bytes)
 - **Word** means 32 bits (four bytes)
- Most ARM's implement two instruction sets
 - 32-bit ARM Instruction Set
 - 16-bit Thumb Instruction Set
- Jazelle cores can also execute Java bytecode (not available in TDMI)

- **When the processor is executing in ARM state:**
 - All instructions are 32 bits wide
 - All instructions must be word aligned
 - Therefore the **pc** value is stored in bits [31:2] with bits [1:0] undefined (as instruction cannot be halfword or byte aligned).

- **When the processor is executing in Thumb state:**
 - All instructions are 16 bits wide
 - All instructions must be halfword aligned
 - Therefore the **pc** value is stored in bits [31:1] with bit [0] undefined (as instruction cannot be byte aligned).

- **When the processor is executing in Jazelle state:(not available in TDMI)**
 - All instructions are 8 bits wide
 - Processor performs a word access to read 4 instructions at once
 - ARM bit Advanced RISC Machine **T** - Thumb architecture extension Two separate instruction sets, 32-bit ARM instructions and 16-bit Thumb instructions **D** - Debug extension **M** - Enhanced multiplier **I** - Embedded ICE macrocell extension

Processor States

- State determines **which instruction set being executed**
- **3 instruction sets** : ARM, Thumb and Jazelle
- **ARM state** – 32 bit instructions
- **Thumb state** – 16 bit instructions
- **Jazelle state** – 8 bit instructions, over 60 % of java bytecodes are implemented in hardware, rest of the codes are implemented in software

	ARM (cpsr T=0)	Thumb (cpsr T =1)
Instruction Size	32 bit	16 bit
Core instructions	58	30
Conditional execution	most	Only branch instructions
Data processing instructions	Access to barrel shifter and ALU	Separate barrel shifter and ALU instructions
Program status register	Read write in privileged mode	No direct access
Register usage	15 GPR + PC	8 GPR+7 high register+PC